

Python Pattern Matching and Regular Expressions

Regular Expressions - regex

Regular expressions are very powerful pattern matching.

Uses the *re* library.

Regex'es are about matching patterns and extracting matches.

Full and complex documentation at

<https://docs.python.org/3/library/re.html>

The general form of a regex is:

```
result = re.method(pattern, string_to_search_in)
```

method is here a method from the *re* library.

The following slides will explain what the *pattern* is, but it should be a raw string in order to avoid string interpretation (like `\n` means newline in a string). A raw string is completely literal – what you see is what you get. A string is made raw by putting an **r** in front.

Example: `r'^ID\s+(\w+)'`

Regex Patterns – Classes – what is the char

Any simple chars just matches themselves.

Built-in character classes:

- `\s` matches a whitespace
- `\S` matches a non-whitespace
- `\d` matches a digit
- `\D` matches a non-digit
- `\w` matches a wordchar which is a-zA-Z0-9_
- `\W` matches a non-wordchar
- `\n` matches newline
- `.` 1 char, matches anything but newline
- `\.` escaping (backslash) a special char gives normal char

Make your own classes with `[]`

`[aB4-6]` matches only one of the chars aB456

`[^xY]` matches anything but x and Y

Regex Patterns – Quantifiers – how many

A single simple char just matches itself, but a quantifier can be added to determine how many times.

- ? Zero or one time
- + One or more times
- * Zero or more times

The {} can be used to make a specific quantification.

- {4} Four times
- {,3} At most three times
- {5,} Minimum five time
- {3,5} Between three and five times

Quantifiers are greedy, can be made non-greedy with extra ?

A few examples:

- A{3,4}C? Match AAAA AAA AAAAC AAAC
- \s\w{4}\s Match any four-letter word in a sentence

Regex Patterns – Groups – what goes together

The parenthesis denote a group. A group belongs together.

Example:

ABC(xyz)?DEF matches both ABCDEF and ABCxyzDEF

Either the entire group xyz is matched once or not (?)

The content of the group can be captured, see later.

Non-capturing group (?:)

The pipe sign | means **or**

A(BC|DEF)G matches either ABCG or ADEFG

Other special chars

^ Must be first, bind a match to the start of line

\$ Must be last, bind a match to the end of line

\b Word-boundary (whitespace, comma, BoL, EoL, etc) 0-span

Benefits of using a regex

Regular expressions are often used in loops.

```
mymatchobject = re.search(pattern, string)
```

The match object can be investigated for matches - capturing

<code>mymatchobj.group(0)</code>	# Entire match
<code>mymatchobj.group(1)</code>	# First group
<code>mymatchobj.start(1)</code>	# Start of first group in string
<code>mymatchobj.end(1)</code>	# End of first group in string

Examples

Testing if there is a match.

```
myStr = 'In this string is an accession AB232346 somewhere'
if re.search(r"\b[A-Z]{1,2}\d{6,8}\b", myStr) is not None:
    print('Yeah, there is an accession number somewhere')
```

Capturing a match, notice parenthesis.

```
myStr = 'In this string is an accession AB232346 somewhere'
result = re.search(r"\b([A-Z]{1,2}\d{6,8})\b", myStr)
if result is None:
    print('No match')
else:
    print("I got one", result.group(1))
```

Examples

Input is a swissprot file. Job is to find the first accession number on the first AC line. This is a fairly typical way to use a regular expression when parsing a file.

```
acc = None
for line in infile:
    if acc is None:
        result = re.search(r'^AC\s+(\w+);', line):
        if result is not None:
            acc = result.group(1)
print("Accession number:", acc)
```

Achieving the same in a different manner.

```
acc = None
for line in infile:
    if acc is None and line.startswith('AC'):
        acc = re.search(r'^AC\s+(\w+);', line).group(1):
print("Accession number:", acc)
```

Commonly used methods of the re library

Find a match anywhere in the string.

```
result = re.search(pattern, string)
```

Find a match only in the beginning of the string.

```
result = re.match(pattern, string)
```

Split string on a pattern.

```
resultList = re.split(pattern, string)
```

Return all matches as list of strings.

```
resultList = re.findall(pattern, string)
```

Return string where matches are replaced with replacement string.

Count = 0 means all occurrences.

```
resultString = re.sub(pattern, replacement, string, count=0)
```

The last method is surprisingly useful and versatile.